10.9.2026

Practical - 6

Hamming code

AIM:

Write program to implement error detection and correction using hamming code concept. Make test run to input data stream and verify correction feature.

CODE:

```
def calc_redundant_bits(m):
    for r in range(m+1):
        if (2**r) >= (m+r+1):
            return r

def main():
    data = input("Enter data bits: ").strip()
    m = len(data)
    r = calc_redundant_bits(m)
    n = m+r
    print(f"Number of r needed: {r}")
    print(f"Total frame length: {n}")
    code = ['0'] * (n+1)
    j = 0
    for i in range(1, n+1):
        if (i & (i-1)) != 0:
            code[i] = data[-(j+1)]
            j += 1
    for i in range(r):
        parity_pos = 2**i
        val = 0
        for pos in range(1, n+1):
            if (pos & parity_pos) != 0 and pos != parity_pos:
                if code[pos] == '1':
                    val ^= 1
        code[parity_pos] = str(val)
```

```
transmitted_str = "".join(code[1:][::-1])
print(f"Data to be transmitted: {transmitted_str}")
change = input("\n Transmitted bit change").strip().
                                          upper()
if change == "YES":
    received_str = input(f"Enter corrupted data").strip()
    if len(received_str) != n:
        print(f"Error: Received data be exactly {n}
                    bits long!")
        return
    received_code = ['0'] + list(received_str[::-1])
else:
    received_code = list(code)

error position = 0
for i in range(r):
    parity_pos = 2 ** i
    val = 0
    for pos in range(1, n+1):
        if (pos & parity_pos) != 0:
            if received_code[pos] == '1':
                val ^= 1
    if val != 0:
        error_position += parity_pos

if error_position == 0:
    print("\n Verification Result: No error detected")
else:
    print("\n Verification Result: Error detected!")
    print(f"Wrong bit at position: {error_position}")
    received_code[error_position] = '0' if received_code
        [error_position] == '1' else '1'
    corrected_str = "".join(received_code[1:][::-1])
    print(f"Corrected data stream: {corrected_str}")

if __name__ == "__main__":
    main()
```

OUTPUT:

①

Enter data bits : 101100

Number of redundant bits (r) needed : 4

Total frame length (m+r) : 10

Data to be transmitted : 1011100011

Transmitted bit change? (YES/NO): YES

Enter the corrupted data stream (10 bits): 1011101011

Verification Result : Error detected!

Wrong bit is at position : 9

Corrected data stream: 1011100011

②

Enter data bits : 1011

Number of redundant bits (r) needed : 3

Total data frame length (m+r) : 7

Data to be transmitted : 1010101

Transmitted bit change? (YES/NO): NO

Verification result: No error detected in transmission

RESULT:

Thus the Hamming code was implemented successfully for error detection and correction.